AI Cloud Lecture Organizer & Search Platform

Phase 1：需求分析與系統架構規劃書（Phase 1 V5.3 嚴謹工程修正版）

> **專案名稱**：AI 雲端多格式講義管理與個人知識檢索平台

> **英文名稱**：AI Cloud Multi-format Personal Knowledge Management & Retrieval Platform

> **系統定位**：Multi-format Personal Knowledge Management & AI Retrieval Platform

> **版本**：Version 5.3（Fencing Token 生命週期統一、複合外鍵跨租戶隔離、狀態機退避持久化與 44 項測試矩陣交付版）

> **當前階段**：Phase 1 需求分析與系統架構規劃（無實質專案程式碼變更，保持工作目錄乾淨）

> **目標平台**：Render.com (單一 Web Service + Render PostgreSQL + Persistent Storage)

---

1. 專案目標與核心需求（Project Vision & Core Requirements）

1.1 核心專案目標

本專案為大學資訊相關課程專題而設計，旨在提供一個高效、穩定、架構清晰且具備長期擴充能力的「AI 雲端多格式講義管理與個人知識檢索平台」。系統以**長期原始檔案持久化保存**為基石，透過統一的**解析適配層（Parser Adapter Layer）**，抽取多元格式檔案之文字與精確結構定位資訊，轉換為**統一內容資料格式（Unified Content Representation）**，並深度整合 PostgreSQL 內建全文搜尋（FTS）、RAG 自然語言問答及具備樂觀鎖控制的 Markdown 個人筆記系統：

1. **原始檔案長期持久化保存與管理**：使用者上傳的原始檔案永久保存，支援隨時預覽、安全下載、與個人筆記並排閱讀。
2. **多格式講義自動解析與結構化定位**：正式支援 PDF、DOCX、TXT、MD、CSV、XLSX、PPTX，並在架構上預留圖片 OCR 與影音轉錄擴充能力。
3. **無損更新與安全重新解析（Reprocess）**：當 Parser 版本升級或 Chunking 規則更新時，系統能調用持久化的原始檔案重新解析，並透過「Fencing Token 版本控制」與「原子性替換機制」確保舊資料不被破壞、舊 Worker 不會覆蓋新結果。
4. **多維度內容自動整理**：自動產生涵蓋整體摘要、章節/工作表要點、關鍵字與核心概念的知識整理。

5. **降低幻覺並提供可驗證來源（Verifiable Grounding）**：回答嚴格依據檢索到的片段，並由後端查核真實來源文件、頁碼、工作表列號或投影片標題；檢索資料不足時如實告知。
6. **講義與個人筆記並排研讀（Split View）**：左側閱覽原始檔案或解析文字與結構定位，右側編輯 Markdown 筆記，具備 Debounce 自動儲存與版本衝突防護。
7. **AI 輔助筆記（預覽後確認機制）**：一鍵轉存摘要或由 AI 補充觀念，強制彈出預覽比對，由使用者手動確認後寫入，絕不直接覆蓋既有筆記。
8. **無鎖定、高安全性的 AI 串接**：支援 OpenAI 相容協定（OpenAI、Groq、DeepSeek 等），金鑰僅暫存於當前工作階段記憶體，不持久化於前端儲存或後端硬碟，並具備嚴密的 SSRF（Undici IP Pinning）防護。

1.2 嚴格邊界與技術限制（Scope Boundaries）

- **固定技術基線（維持不變）**：
- 前端：React + Vite + 原生 CSS（嚴禁 TypeScript, Next.js, Vue, Angular, Tailwind, Bootstrap）。
- 後端：Node.js + Express（嚴禁 Laravel, Django, Spring Boot, NestJS）。
- 資料庫：PostgreSQL + 原生 `pg` 連線池（嚴禁 ORM, SQLite）。
- 搜尋引擎：PostgreSQL Full-Text Search + 中文 N-gram（嚴禁向量資料庫、Embedding API、Elasticsearch）。
- 基礎設施：單一 Render Web Service + Render PostgreSQL（嚴禁 Redis, BullMQ, Kubernetes, Docker, 微服務）。
- **語言與文字處理限制**：
- 第一版全文檢索不實作自動繁簡中文轉換，使用者搜尋時須以檔案原本字系為主。

---

2. 完整系統架構（System Architecture）

本系統採用單一 Render Web Service 架構，前後端同源通訊，職責清晰分離：

```
flowchart TD
    subgraph Browser ["使用者瀏覽器 (Client)"]
        UI["React SPA (原生 CSS + Vite 打包)"]
        NoteEditor["Markdown 筆記編輯器 (Debounce Autosave & 樂觀鎖)"]
        SplitView["檔案預覽與筆記並排閱讀 (Split View)"]
        MemoryStore["Session 記憶體 (暫存 AI API Key，不持久化)"]
    end

    subgraph RenderApp ["Render.com Web Service (單一服務)"]
        Express["Express.js HTTP 伺服器 (PORT: 0.0.0.0)"]
        StaticServe["靜態檔案中介層 (託管 React 建置成果 client/dist)"]
        AuthMid["身份認證與 Session 檢查中介層 (CSRF + Origin)"]
        FileValidator["檔案上傳防護 (MIME/Magic/ZipBomb/XXE/路徑安全)"]
        StorageAdapter["統一儲存適配層 (Persistent Storage Adapter)"]
        ParserRegistry["統一解析適配層 (Unified Parser Registry)"]

        subgraph Parsers ["Parser Adapters"]
            PDFParser["PDF Parser (pdfjs-dist)"]
            DocxParser["DOCX Parser (mammoth)"]
            TextParser["TXT/MD Parser (串流讀取)"]
            SheetParser["XLSX/CSV Parser (xlsx / csv-parser 串流)"]
            SlideParser["PPTX Parser (adm-zip + xml2js)"]
            FutureParsers["擴充預留: OCR / 音訊轉錄"]
        end

        TaskWorker["通用任務處理器 (FOR UPDATE SKIP LOCKED + Fencing Token + next_retry_at)"]
        Tokenizer["中文 2-gram 與英數分詞器"]
        APIProxy["AI API 轉發代理器 (Undici Dispatcher + IP Pinning SSRF 防護)"]
        DBPool["PostgreSQL 連線池 (pg.Pool, SSL 憑證驗證)"]
    end

    subgraph PersistentStorage ["持久化儲存 (Persistent Storage)"]
        DiskStore["Render Persistent Disk / S3 相容儲存 (原始檔案永久儲存)"]
    end

    subgraph CloudDB ["Render PostgreSQL Database"]
        PG_Session["系統表: session"]
        PG_Business["業務表: users, courses, documents, chunks, summaries, notes, chat (複合外鍵強制隔
離)"]
    end

    subgraph ExternalAI ["使用者自訂 AI API (External Provider)"]
        LLM["OpenAI-compatible LLM 服務"]
    end

    UI -->|"HTTP 請求 (同源 /api/*)"| Express
    MemoryStore -->|"傳送暫時性 API Key (Header / Payload)"| Express
    Express --> StaticServe
    Express --> AuthMid
    AuthMid --> FileValidator
    FileValidator --> StorageAdapter
    StorageAdapter <-->|"儲存 / 讀取原始檔案 (SHA-256 驗證)"| DiskStore
    StorageAdapter --> TaskWorker
    TaskWorker --> ParserRegistry
    ParserRegistry --> Parsers
    Parsers --> Tokenizer
    Tokenizer --> DBPool
```

```
    AuthMid --> DBPool
    DBPool <-->|"SQL 查詢 + FTS 檢索"| CloudDB
    AuthMid --> APIProxy
    APIProxy <-->|"HTTPS 轉發 (遮蔽 Log & IP Pinning)"| LLM
```

3. 多格式檔案管理與支援矩陣（Multi-format Support Matrix）

檔案類別	格式清單	抽取內容與定位資訊	支援狀態	採用技術與安全性考量
第一類：文件格式	`PDF`	頁碼、文字區塊、標題	**第一版正式支援**	`pdfjs-dist`（純 JS，逐頁抽取純文字，安全無二進位執行風險）
	`DOCX`	段落編號、標題路徑、內文	**第一版正式支援**	`mammoth`（純 JS，安全抽取 raw text 與 heading，過濾巨集）
	`TXT`, `MD`	行號、章節標題、層級	**第一版正式支援**	Node.js 原生串流讀取，支援大檔串流與 Markdown 結構切分
	`DOC`, `RTF`, `ODT`	段落文字	後續階段擴充	舊版二進位 DOC 格式結構複雜，規劃後續沙箱轉檔
第二類：試算表格式	`CSV`	列號、欄位標頭、單元格資料	**第一版正式支援**	`csv-parser`（串流解析，自動嗅探編碼與分隔符號，記憶體開銷低）
	`XLSX`	工作表名稱、欄位名稱、列號、儲存格範圍	**第一版正式支援**	`xlsx`（純文字快取讀取，不執行公式與外部連線，防範巨大工作表）
	`XLS`, `ODS`	工作表名稱、列號、儲存格	後續階段擴充	評估記憶體開銷與安全沙箱
第三類：簡報格式	`PPTX`	投影片編號、投影片標題、文字方塊	**第一版正式支援**	`adm-zip` + `xml2js`（安全解析 slide XML，防 Zip Slip 與 XXE）
	`PPT`, `ODP`	投影片編號、文字區塊	後續階段擴充	舊版二進位 PPT 格式結構複雜，列後續階段
第四類：圖片格式	`JPG`, `PNG`, `WEBP`	圖片文字、區域座標、置信度	後續階段擴充	區分一般圖片與掃描文件，規劃 Tesseract.js 或自訂 Vision API
	`TIFF`, `HEIC`	OCR 文字內容	架構預留	需額外解碼轉檔，預留介面
第五類：影音格式	`MP3`, `WAV`, `M4A`	語音轉錄文字、時間戳記、講者	架構預留	預留 Whisper-compatible API 轉錄介面，需使用者手動授權
	`MP4`, `MOV`, `WEBM`	字幕軌道、時間區間、轉錄文字	架構預留	抽取內嵌 SRT/VTT 字幕，或音訊軌轉錄
第六類：其他格式	`JSON`, `XML`, `EPUB`	鍵值對文本、結構節點、章節	架構預留	預留擴充 Parser
純儲存格式	不支援解析之二進位檔	僅儲存基本檔案資訊，標註不可解析	僅保存不解析	允許使用者長期存放下載，但不進入全文檢索與 RAG 管道

4. 原始檔案持久化、備份與一致性保證（Persistent Storage & Data Consistency）

4.1 雙模式儲存適配層（Dual-Mode Storage Adapter）

系統設計統一抽象介面 `StorageAdapter`，透過環境變數 `STORAGE_DRIVER=local|s3` 切換：

1. **Local Disk 模式（Render Persistent Disk 掛載）**：

- 適用於 Render 付費方案掛載之 Disk（掛載路徑如 `/var/data/storage`）或本機開發。
- 檔案儲存以 `users/{userId}/{documentId}/{uuid}.ext` 隔離，避免路徑穿越。

1. **S3 相容 Object Storage 模式（雲端多實例擴展）**：

- 支援 Cloudflare R2、AWS S3。透過 S3 Client 進行串流讀寫，解除單一 Disk 掛載限制。

4.2 SHA-256 Checksum 完整性驗證機制

- **上傳完成**：計算檔案之 SHA-256 雜湊，寫入 `documents.checksum_sha256`。
- **解析前校驗**：任何 Worker 開始解析（含 Retry 與 Reprocess）前，由 Storage 讀取檔案並重新計算雜湊比對。若不一致，立即標記為 `failed` 並記錄 `FILE_CORRUPTED` 拒絕執行。
- **下載驗證**：客戶端下載時於 Response Header 提供 `ETag` (SHA-256)，供客戶端核對。
- **Reupload**：使用者重新上傳後，重新計算並覆蓋 `checksum_sha256`。

4.3 備份、復原目標與災難復原演練（RPO / RTO）

1. **目標指標**：

- **RPO（Recovery Point Objective，復原點目標）**：**24 小時**（每日自動冷備份）。
- **RTO（Recovery Time Objective，復原時間目標）**：**2 小時**（包含資料庫恢復與檔案鏡像還原）。

1. **資料庫備份**：

- 每日凌晨排程執行 `pg_dump`，匯出 SQL 結構與資料並進行 Gzip 壓縮與 AES-256 加密，保留 30 天滾動覆蓋。

1. **原始檔案備份**：

- Local Disk 模式：每日排程透過 `rsync` 鏡像同步至異地獨立儲存空間。
- S3 模式：開啟 S3 原生 Bucket Versioning（多版本控制）與生命週期管理。

1. **災難復原演練（Disaster Recovery Drill）**：

- 每半年進行一次還原演練。復原時，先恢復 PostgreSQL 資料表，再將對應日期的原始檔案同步至儲存路徑。
- 執行系統一致性檢查腳本 `npm run verify:integrity`，遍歷 DB 中所有 `documents`，確認實體檔案存在且 Checksum 100% 吻合。

4.4 DB 與 Storage 一致性與補償機制（P1-4 規範）

- **暫存檔與正式檔命名規範**：
- 暫存檔：`tmp/upload_{randomUUID}.tmp`。
- 正式檔：`storage/users/{userId}/{documentId}/{randomUUID}.{ext}`。
- **上傳補償機制**：
- 先寫入暫存檔，校驗通過後移至正式路徑，取得 Checksum。
- 執行 DB INSERT。若 DB 拋出異常，立即執行非同步補償刪除該正式實體檔案。

- **刪除補償機制**：
- 先在 DB 交易內執行 `DELETE FROM documents WHERE id = $id AND user_id = $userId`。
- DB 提交成功後，再調用 `StorageAdapter.delete(storage_path)`。若實體刪除失敗，寫入系統孤兒待清理日誌。
- **孤兒檔案排程掃描（Orphan Cleanup Cron）**：
- 伺服器每週執行一次清理排程，比對 Storage 實際檔案列表與 DB 中的 `storage_path`，清除所有未在 DB 登記且建立超過 24 小時的孤兒檔案。

---

5. 統一 Parser Adapter 介面與大檔串流架構（P1-2 規範）

為避免將數十 MB 的檔案一次性完整讀入記憶體造成 Node.js 記憶體耗盡（OOM），統一 Parser Adapter 介面全面支援**檔案句柄（File Handle）與串流讀取**：

5.1 檔案輸入句柄（Input Handle）規範

```
/**
 * @typedef {Object} ParserInputHandle
 * @property {string} filePath - 實體磁碟路徑 (Local Disk 模式)
 * @property {number} fileSize - 檔案位元組大小
 * @property {function(): NodeJS.ReadableStream} getStream - 取得檔案讀取串流
 * @property {function(): Promise<Buffer>} getBuffer - 針對小型檔案取得完整 Buffer (有大小上限限制)
 */

/**
 * @typedef {Object} ParserContext
 * @property {string} documentId - 文件 UUID
 * @property {string} userId - 使用者 UUID
 * @property {string} filename - 原始檔案名稱
 * @property {AbortSignal} signal - 任務取消信號 (用於中斷長任務)
 */
```

5.2 統一 Parser 輸出契約（JSDoc 規範）

```
/**
 * @typedef {Object} UnifiedChunk
 * @property {number} chunkIndex - 分塊序號（從 0 起算）
 * @property {string} content - 分塊純文字
 * @property {Object} sourceLocation - 結構化定位物件（須通過 JSON Schema 驗證）
 * @property {Object} [metadata] - 額外格式元數據
 */

/**
 * @typedef {Object} UnifiedExtractionResult
 * @property {string} parserType - 解析器識別碼（如 'pdf', 'docx', 'xlsx', 'txt', 'md'）
 * @property {string} parserVersion - 解析器版本（如 '1.0.0'）
 * @property {'completed'|'partial_success'|'failed'} extractionStatus - 抽取狀態
 * @property {string} rawText - 全文純文字內容
 * @property {Array<UnifiedChunk>} chunks - 分塊清單
 * @property {Object} metadata - 格式特有元數據（頁數、工作表清單等）
 * @property {Array<string>} warnings - 警告訊息（如略過空白頁面）
 * @property {Array<string>} errors - 錯誤訊息清單
 */
```

5.3 `BaseParser` 抽象類別

```
class BaseParser {
  constructor(parserType, parserVersion) {
    this.parserType = parserType;
    this.parserVersion = parserVersion;
    this.supportedMimeTypes = [];
    this.supportedExtensions = [];
  }

  canParse(mimeType, extension) {
    const ext = extension.startsWith('.') ? extension.slice(1).toLowerCase() : extension.toLowerCase();
    return this.supportedMimeTypes.includes(mimeType) || this.supportedExtensions.includes(ext);
  }

  /**
   * @param {ParserInputHandle} inputHandle 檔案輸入句柄（支援串流與 Buffer）
   * @param {ParserContext} context 上下文資訊與取消信號
   * @returns {Promise<UnifiedExtractionResult>}
   */
  async parse(inputHandle, context) {
    throw new Error('parse() must be implemented by concrete parser');
  }
}
```

---

6. 正式 Source Location JSON Schema（含 TXT / MD 補齊，P1-1 規範）

為防止各 Parser 欄位混亂，使用標準 JSON Schema（Draft-07）嚴格規範 8 種格式之定位結構，**全面強制禁止未定義屬性（`additionalProperties: false`）**：

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "SourceLocationSchema",
  "type": "object",
  "oneOf": [
    {
      "title": "PDFLocation",
      "type": "object",
      "required": ["type", "pageNumber"],
      "additionalProperties": false,
      "properties": {
        "type": { "const": "pdf" },
        "pageNumber": { "type": "integer", "minimum": 1 },
        "bbox": {
          "type": "array",
          "items": { "type": "number" },
          "minItems": 4,
          "maxItems": 4
        }
      }
    },
    {
      "title": "DOCXLocation",
      "type": "object",
      "required": ["type", "paragraphIndex", "headingPath"],
      "additionalProperties": false,
      "properties": {
        "type": { "const": "docx" },
        "paragraphIndex": { "type": "integer", "minimum": 0 },
        "headingPath": { "type": "string", "minLength": 1 }
      }
    },
    {
      "title": "TXTLocation",
      "type": "object",
      "required": ["type", "startLine", "endLine"],
      "additionalProperties": false,
      "properties": {
        "type": { "const": "txt" },
        "startLine": { "type": "integer", "minimum": 1 },
        "endLine": { "type": "integer", "minimum": 1 },
        "startChar": { "type": "integer", "minimum": 0 },
        "endChar": { "type": "integer", "minimum": 0 }
      }
    },
    {
      "title": "MDLocation",
      "type": "object",
      "required": ["type", "startLine", "endLine"],
      "additionalProperties": false,
      "properties": {
        "type": { "const": "md" },
        "startLine": { "type": "integer", "minimum": 1 },
        "endLine": { "type": "integer", "minimum": 1 },
        "headingPath": { "type": "string" },
        "sectionLevel": { "type": "integer", "minimum": 1, "maximum": 6 }
      }
```

```
    },
    {
      "title": "XLSXLocation",
      "type": "object",
      "required": ["type", "sheetName", "rowIndex", "cellRange"],
      "additionalProperties": false,
      "properties": {
        "type": { "const": "xlsx" },
        "sheetName": { "type": "string", "minLength": 1 },
        "rowIndex": { "type": "integer", "minimum": 1 },
        "cellRange": { "type": "string", "minLength": 2 },
        "columnName": { "type": "string" }
      }
    },
    {
      "title": "CSVLocation",
      "type": "object",
      "required": ["type", "rowIndex"],
      "additionalProperties": false,
      "properties": {
        "type": { "const": "csv" },
        "rowIndex": { "type": "integer", "minimum": 1 },
        "columnName": { "type": "string" },
        "columnIndex": { "type": "integer", "minimum": 0 }
      }
    },
    {
      "title": "PPTXLocation",
      "type": "object",
      "required": ["type", "slideNumber"],
      "additionalProperties": false,
      "properties": {
        "type": { "const": "pptx" },
        "slideNumber": { "type": "integer", "minimum": 1 },
        "shapeIndex": { "type": "integer", "minimum": 0 },
        "slideTitle": { "type": "string" }
      }
    },
    {
      "title": "ImageLocation",
      "type": "object",
      "required": ["type", "boundingBox"],
      "additionalProperties": false,
      "properties": {
        "type": { "const": "image" },
        "boundingBox": {
          "type": "array",
          "items": { "type": "number" },
          "minItems": 4,
          "maxItems": 4
        },
        "confidence": { "type": "number", "minimum": 0, "maximum": 1 }
      }
    },
    {
      "title": "AudioLocation",
      "type": "object",
      "required": ["type", "startTime", "endTime"],
```

```
      "additionalProperties": false,
      "properties": {
        "type": { "const": "audio" },
        "startTime": { "type": "string", "pattern": "^\\d{2}:\\d{2}:\\d{2}(\\.\\d+)?$" },
        "endTime": { "type": "string", "pattern": "^\\d{2}:\\d{2}:\\d{2}(\\.\\d+)?$" },
        "speaker": { "type": "string" }
      }
    },
    {
      "title": "VideoLocation",
      "type": "object",
      "required": ["type", "startTime", "endTime"],
      "additionalProperties": false,
      "properties": {
        "type": { "const": "video" },
        "startTime": { "type": "string", "pattern": "^\\d{2}:\\d{2}:\\d{2}(\\.\\d+)?$" },
        "endTime": { "type": "string", "pattern": "^\\d{2}:\\d{2}:\\d{2}(\\.\\d+)?$" },
        "subtitleIndex": { "type": "integer", "minimum": 0 }
      }
    }
  ]
}
```

---

7. 試算表（XLSX / CSV）與檔案限制規格（P1-7 規範）

7.1 全域檔案與解析資源限制清單

格式分類	最大上傳大小	最大解壓縮大小	壓縮比上限	結構邊界上限	記憶體上限	執行逾時
PDF	15 MB	不適用	不適用	上限 500 頁	256 MB	60 秒
DOCX	15 MB	100 MB	20 : 1	上限 1,000 個 XML 條目	256 MB	60 秒
XLSX	10 MB	50 MB	20 : 1	單表上限 20,000 列 / 100 欄	256 MB	60 秒
PPTX	20 MB	100 MB	20 : 1	上限 200 頁投影片	256 MB	60 秒
TXT / MD	10 MB	不適用	不適用	上限 50,000 行文字	128 MB	30 秒
CSV	10 MB	不適用	不適用	上限 50,000 列 (串流解析)	128 MB	30 秒

7.2 XLSX / CSV 解析細節

- **XLSX**：公式僅讀取快取值（`cell.w` / `cell.v`），若無快取值保留公式字串，**絕不重新計算、不執行巨集**；合併儲存格文字僅綁定首格，**絕不重複寫入多個 Chunk**；隱藏工作表預設略過。
- **CSV**：自動偵測 UTF-8 BOM / UTF-8 / UTF-16LE / Big5 編碼與分隔符號（`,`、`;`、`\t`）；支援雙引號跳脫與換行儲存格；不規則欄位補空或截斷；大檔全面採用 `csv-parser` 串流管線處理。

---

8. Reprocess 版本生命週期與 Fencing Token 原子替換（P0-1 核心規範）

8.1 統一 `processing_version`（Fencing Token）生命週期

為徹底解決第 8 章與第 9 章版本遞增時機不一致的矛盾，明確定義 **`processing_version` 的唯一生命週期**：

1. **初次上傳建立文件**：`processing_version = 1`，狀態設為 `pending`，`worker_id = NULL`，`retry_count = 0`。
2. **Reprocess / Retry 觸發時**：

- 使用者發起 Reprocess 或系統判定可重試時，僅將 `status` 改為 `pending`，清除 `error_message`。**此時不遞增版本**。

1. **Worker 成功認領任務時（唯一遞增點！）**：

- **由原子認領 SQL 執行唯一的版本遞增：`processing_version = processing_version + 1`**。
- 這個新版本號即為本次任務執行的 **Fencing Token**，代表唯一的合法處理世代（Generation）。

1. **Lease 過期接管時**：

- 若前任 Worker 超時（`lease_timeout_at < NOW()`），新 Worker 執行認領時，`processing_version` 再次 `+ 1`。前任 Worker 的 Token 立即失效。

1. **最終寫入提交時**：

- Worker 提交時強制校驗資料庫中的 `processing_version` 是否等於自己的 Token。若已被接管（版本已被推進），寫入直接失敗回滾，絕無覆蓋新結果之可能。

```
sequenceDiagram
    autonumber
    actor User as 使用者
    participant WorkerB as 新 Worker B (Parser 1.2)
    participant WorkerA as 舊 Worker A (Parser 1.1)
    participant DB as PostgreSQL
    participant Storage as Persistent Storage

    Note over DB: 文件處於 completed 狀態 (processing_version = 1)
    User->>DB: POST /api/documents/:id/reprocess (狀態轉為 pending)

    WorkerB->>DB: 原子認領任務 (UPDATE ... processing_version = 2 RETURNING ...)
    Note over WorkerB: Worker B 取得 Fencing Token = 2

    WorkerA->>DB: 延遲嘗試認領同一任務
    Note over DB: 任務已被 Worker B 鎖定持有且租約有效<br/>Worker A 無法認領 (SKIP LOCKED 略過)

    WorkerB->>Storage: 讀取原始檔案並完成解析 (新 Chunks 暫存於記憶體)

    Note over WorkerB, DB: 關鍵原子替換階段 (ACID Transaction)
    WorkerB->>DB: BEGIN TRANSACTION
    WorkerB->>DB: SELECT processing_version, worker_id FROM documents WHERE id = $docId FOR UPDATE
    Note over WorkerB, DB: 驗證通過：DB 內 processing_version === 2 且 worker_id === WorkerB
    WorkerB->>DB: DELETE FROM document_chunks WHERE document_id = $docId
    WorkerB->>DB: 批次 INSERT INTO document_chunks (新 Chunks)
    WorkerB->>DB: UPDATE documents SET status = 'completed', parser_version = '1.2.0',<br/>reprocess_co
unt = reprocess_count + 1 WHERE id = $docId AND processing_version = 2
    WorkerB->>DB: COMMIT (新解析結果正式生效)
```

8.2 原子替換核心 SQL

```
BEGIN;

-- 1. 鎖定文件主表並核對 Fencing Token 與 Worker 所有權
SELECT id, processing_version, worker_id, lease_timeout_at, status
FROM documents
WHERE id = $docId
FOR UPDATE;

-- 後端檢驗邏輯：
-- if (row.status === 'cancelled' || row.processing_version !== myProcessingVersion ||
--     row.worker_id !== myWorkerId || new Date(row.lease_timeout_at) < new Date()) {
--     ROLLBACK; return; -- 舊 Worker 或過期 Worker 立即放棄
-- }

-- 2. 刪除該文件之既有舊 Chunks
DELETE FROM document_chunks WHERE document_id = $docId;

-- 3. 批次插入新 Chunks (確保符合 uq_chunks_doc_chunk_idx 唯一約束)
INSERT INTO document_chunks (
    document_id, user_id, chunk_index, content, source_location, chunking_version, search_vector
) VALUES ($1, $2, $3, $4, $5, $6, to_tsvector('simple', $7));

-- 4. 更新文件主表版本資訊與狀態
UPDATE documents
SET status = 'completed',
    parser_type = $newParserType,
    parser_version = $newParserVersion,
    chunking_version = $newChunkingVersion,
    reprocess_count = reprocess_count + 1,
    error_message = NULL,
    updated_at = NOW()
WHERE id = $docId AND processing_version = $myProcessingVersion;

COMMIT;
```

- **容錯保證**：若新解析或寫入中途拋出任何異常，立即 `ROLLBACK`，**舊 Chunks 與既有索引 100% 完好無損**，絕不發生資料中斷。

---

9. Worker 狀態機、原子認領與取消重試機制（P0-3 規範）

9.1 Worker 完整狀態機（State Machine）

系統定義 5 種互斥且完備的生命週期狀態，並由 DDL CHECK 約束強制保護：

`CHECK (status IN ('pending', 'processing', 'completed', 'failed', 'cancelled'))`

```
stateDiagram-v2
    [*] --> pending: 檔案上傳成功 / 觸發 Reprocess
    pending --> processing: Worker 成功認領 (Fencing Token + 1)
    processing --> completed: 解析成功，原子性提交
    processing --> failed: 解析異常 / 租約逾期失效
    processing --> cancelled: 使用者主動取消任務
    pending --> cancelled: 使用者主動取消排隊任務
    failed --> pending: 重試冷卻完畢 (next_retry_at 到期)
    completed --> pending: 使用者觸發 Reprocess
    cancelled --> [*]: 終止狀態
    completed --> [*]: 正常就緒狀態
```

9.2 原子性任務認領 SQL（唯一版本遞增）

```
UPDATE documents
SET status = 'processing',
    worker_id = $myWorkerId,
    processing_started_at = NOW(),
    lease_timeout_at = NOW() + INTERVAL '3 minutes',
    processing_version = processing_version + 1, -- 唯一遞增點！
    retry_count = CASE WHEN status = 'failed' THEN retry_count + 1 ELSE retry_count END,
    updated_at = NOW()
WHERE id = (
    SELECT id FROM documents
    WHERE (status = 'pending')
       OR (status = 'failed' AND retry_count < 3 AND (next_retry_at IS NULL OR next_retry_at <= NOW()))
       OR (status = 'processing' AND lease_timeout_at < NOW())
    ORDER BY created_at ASC
    FOR UPDATE SKIP LOCKED
    LIMIT 1
)
RETURNING id, user_id, original_filename, storage_path, file_extension, mime_type, processing_version,
worker_id;
```

9.3 失敗處理與 `next_retry_at` 指數退避持久化

當 Worker 遇到可重試錯誤（如外部 AI 逾時、連線異常）時，執行失敗更新並持久化退避時間：

```
UPDATE documents
SET status = 'failed',
    error_message = $errorMessage,
    lease_timeout_at = NULL,
    -- 指數退避計算：2s, 4s, 8s (加隨機抖動 Jitter 0~1000ms)，上限 30s
    next_retry_at = NOW() + (LEAST(POWER(2, retry_count) * 2, 30) || ' seconds')::interval
                          + (random() * 1000 || ' milliseconds')::interval,
    updated_at = NOW()
WHERE id = $docId AND processing_version = $myProcessingVersion;
```

- **伺服器重啟保護**：退避時間持久化於資料庫 `next_retry_at`。伺服器若重開機，認領 SQL 嚴格比對 `next_retry_at <= NOW()`，**絕不提前重試，防止雷鳴群聚（Thundering Herd）**。
- **永久失敗判定**：若為不可重試錯誤（格式不支援、損壞檔案、XXE），直接設 `retry_count = 3`，`next_retry_at = NULL`，永久停止認領。

9.4 任務取消流程（Cancellation Flow）

- 使用者呼叫 `POST /api/documents/:id/cancel`，後端將狀態更新為 `cancelled`。
- Worker 在心跳更新（每 30 秒）或最終寫入前，檢查目前狀態。若發現 `status === 'cancelled'`，Worker 立即中止處理並執行暫存檔清理，**絕對不提交任何結果**。

---

10. Reupload 原子替換與補償機制（P1-3 規範）

當使用者因講義改版或檔案損壞而執行 **Reupload** 時，絕不可先刪除舊資料再上傳，實作無損原子替換流程：

1. **接收與暫存新檔**：上傳串流寫入暫存路徑 `tmp/upload_new_{uuid}.tmp`。
2. **格式與安全驗證**：校驗 Magic Bytes、MIME、ZIP 核心結構與 XXE，計算新檔案 SHA-256。
3. **持久化預備**：將新檔案搬移至正式目錄的新檔案名稱 `storage/users/{userId}/{docId}/{newUuid}.{ext}`。
4. **解析新內容**：調用 Parser 生成新 Chunks，驗證 Source Location Schema 合法性。
5. **資料庫原子替換交易**：

```
BEGIN;
SELECT storage_path FROM documents WHERE id = $docId AND user_id = $userId FOR UPDATE;
-- 刪除舊 Chunks
DELETE FROM document_chunks WHERE document_id = $docId;
-- 批次插入新 Chunks
INSERT INTO document_chunks (...) VALUES (...);
-- 更新文件主表，記錄新的 storage_path 與 checksum
UPDATE documents
SET storage_path = $newStoragePath,
    checksum_sha256 = $newChecksum,
    file_size = $newFileSize,
    status = 'completed',
    error_message = NULL,
    updated_at = NOW()
WHERE id = $docId;
COMMIT;
```

1. **清理舊實體檔案**：DB 交易成功提交後，異步刪除舊檔案實體 `oldStoragePath`。
2. **失敗保護**：若上述第 2~5 步任一步驟失敗，交易 `ROLLBACK`，立即刪除新檔案實體 `newStoragePath`，**原本的舊檔案、舊 Chunks 與歷史對話完全保留不受影響**！

---

11. 檔案預覽與下載 API 安全規格（P1 修正）

- `GET /api/documents/:id`：取得中繼資料與狀態。
- `GET /api/documents/:id/preview`：在線安全預覽。
- `GET /api/documents/:id/download`：安全下載原始檔案。
- `POST /api/documents/:id/retry`：失敗任務重試。
- `POST /api/documents/:id/reprocess`：原子性重新解析。
- `POST /api/documents/:id/reupload`：原子性重新上傳替換。

- `POST /api/documents/:id/cancel`：取消進行中任務。

安全防護措施

1. **多租戶權限查核**：一律以 `WHERE id = $id AND user_id = $userId` 查詢，禁止前端指定 `storage_path`。
2. **安全標頭**：全面強制配置 `X-Content-Type-Options: nosniff` 與 `Cache-Control: private, no-store`；HTML/SVG 強制配置沙箱 `Content-Security-Policy: default-src 'none'; sandbox`。下載採用 RFC 5987 檔名編碼（`filename*=UTF-8''...`）。
3. **HTTP Range Request 支援**：大型 PDF 預覽支援 `Accept-Ranges: bytes`、`HTTP 206 Partial Content` 與 `HTTP 416 Range Not Satisfiable`。

---

12. SSRF 深度防禦與完整 IP 封鎖清單（P1-9 規範）

12.1 Undici Dispatcher 正式配置

Node.js 18+ 原生 `fetch` 係由 Undici 驅動，配置 `maxRedirections: 0`、`redirect: 'manual'`，遇 HTTP 3xx 立即拒絕跳轉。

12.2 完整 IP 黑名單檢查清單

DNS 解析（`dns.promises.resolve4/resolve6`）取得之**所有 IP**，均必須通過以下完整黑名單檢驗：

- **IPv4 封鎖範圍**：
- `0.0.0.0/8`（本網路）
- `10.0.0.0/8`（私有網路，RFC 1918）
- `100.64.0.0/10`（電信級 NAT / CGNAT，RFC 6598）
- `127.0.0.0/8`（本機環回 Loopback）
- `169.254.0.0/16`（Link-Local 網段，雲端 Metadata）
- `172.16.0.0/12`（私有網路，RFC 1918）
- `192.0.0.0/24`（IETF 協定指派，RFC 6890）
- `192.0.2.0/24`（TEST-NET-1 測試網段，RFC 5737）
- `192.168.0.0/16`（私有網路，RFC 1918）
- `198.18.0.0/15`（基準測試網段，RFC 2544）
- `198.51.100.0/24`（TEST-NET-2 測試網段，RFC 5737）
- `203.0.113.0/24`（TEST-NET-3 測試網段，RFC 5737）
- `224.0.0.0/4`（多播 Multicast）
- `240.0.0.0/4`（保留網段）
- `255.255.255.255/32`（廣播）
- **IPv6 封鎖範圍**：
- `::/128`（未指定位址）
- `::1/128`（本機環回 Loopback）
- `::ffff:0:0/96`（IPv4-mapped IPv6，**後端強制解包還原為 IPv4，重新帶入上述 IPv4 完整黑名單比對**）
- `64:ff9b::/96`（IPv4/IPv6 轉換）

- `100::/64`（Discard-only prefix）
- `2001:db8::/32`（文件測試網段）
- `fc00::/7`（唯一區域位址 Unique Local Address, ULA）
- `fe80::/10`（Link-Local 位址）
- `ff00::/8`（多播 Multicast）

12.3 IP Pinning 連線釘選（防 DNS Rebinding）

在 Undici Agent 的自訂 `connect` handler 中，直接建立 TCP/TLS socket 至已驗證之安全 IP，傳入 `servername: originalHostname` 供 TLS SNI 憑證校驗，確保連線發起與檢查使用的是同一個、已通過安全驗證的 IP。

---

13. 完整資料庫 Schema 與複合外鍵跨租戶隔離（P0-2, P1-5 DDL）

以下為系統標準 PostgreSQL DDL。**全面實作 `(id, user_id)` 複合唯一約束與複合外鍵，在資料庫層級徹底封死跨租戶關聯（IDOR）**：

```
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

-- 1. 系統管理資料表: session
CREATE TABLE IF NOT EXISTS "session" (
  "sid" varchar NOT NULL COLLATE "default",
  "sess" json NOT NULL,
  "expire" timestamp(6) NOT NULL,
  CONSTRAINT "session_pkey" PRIMARY KEY ("sid")
);
CREATE INDEX IF NOT EXISTS "IDX_session_expire" ON "session" ("expire");

-- 2. 業務資料表: users
CREATE TABLE IF NOT EXISTS "users" (
  "id" uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
  "email" varchar(255) UNIQUE NOT NULL,
  "password_hash" varchar(255) NOT NULL,
  "display_name" varchar(100) NOT NULL,
  "created_at" timestamp NOT NULL DEFAULT NOW(),
  "updated_at" timestamp NOT NULL DEFAULT NOW()
);

-- 3. 業務資料表: courses (建立複合 UNIQUE)
CREATE TABLE IF NOT EXISTS "courses" (
  "id" uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
  "user_id" uuid NOT NULL REFERENCES "users"("id") ON DELETE CASCADE,
  "name" varchar(150) NOT NULL,
  "description" text,
  "created_at" timestamp NOT NULL DEFAULT NOW(),
  "updated_at" timestamp NOT NULL DEFAULT NOW(),
  CONSTRAINT "uq_courses_id_user" UNIQUE ("id", "user_id")
);
CREATE INDEX IF NOT EXISTS "idx_courses_user" ON "courses"("user_id");

-- 4. 業務資料表: documents (複合外鍵連線 courses，建立複合 UNIQUE)
CREATE TABLE IF NOT EXISTS "documents" (
  "id" uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
  "user_id" uuid NOT NULL REFERENCES "users"("id") ON DELETE CASCADE,
  "course_id" uuid,
  "title" varchar(255) NOT NULL,
  "original_filename" varchar(255) NOT NULL,
  "file_extension" varchar(20) NOT NULL,
  "mime_type" varchar(100) NOT NULL,
  "file_size" bigint NOT NULL,
  "checksum_sha256" varchar(64) NOT NULL,
  "storage_path" varchar(512) NOT NULL,
  "status" varchar(30) NOT NULL DEFAULT 'pending'
           CHECK (status IN ('pending', 'processing', 'completed', 'failed', 'cancelled')),
  "parser_type" varchar(50) NOT NULL DEFAULT 'unknown',
  "parser_version" varchar(30) NOT NULL DEFAULT '1.0.0',
  "chunking_version" varchar(30) NOT NULL DEFAULT '1.0.0',
  "processing_version" integer NOT NULL DEFAULT 1,
  "retry_count" integer NOT NULL DEFAULT 0,
  "reprocess_count" integer NOT NULL DEFAULT 0,
  "error_message" text,
  "extraction_metadata" jsonb NOT NULL DEFAULT '{}'::jsonb,
  "worker_id" varchar(64),
  "processing_started_at" timestamp,
```

```
  "lease_timeout_at" timestamp,
  "next_retry_at" timestamp,
  "created_at" timestamp NOT NULL DEFAULT NOW(),
  "updated_at" timestamp NOT NULL DEFAULT NOW(),
  CONSTRAINT "uq_documents_id_user" UNIQUE ("id", "user_id"),
  CONSTRAINT "fk_documents_course_user" FOREIGN KEY ("course_id", "user_id")
              REFERENCES "courses"("id", "user_id") ON DELETE SET NULL
);
CREATE INDEX IF NOT EXISTS "idx_documents_user_course" ON "documents"("user_id", "course_id");
CREATE INDEX IF NOT EXISTS "idx_documents_queue" ON "documents"("status", "lease_timeout_at", "retry_co
unt", "next_retry_at");

-- 5. 業務資料表: document_chunks (複合外鍵連線 documents，加入唯一約束)
CREATE TABLE IF NOT EXISTS "document_chunks" (
  "id" uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
  "document_id" uuid NOT NULL,
  "user_id" uuid NOT NULL,
  "chunk_index" integer NOT NULL,
  "content" text NOT NULL,
  "source_location" jsonb NOT NULL,
  "page_number" integer,
  "chunking_version" varchar(30) NOT NULL DEFAULT '1.0.0',
  "search_vector" tsvector,
  "created_at" timestamp NOT NULL DEFAULT NOW(),
  CONSTRAINT "uq_chunks_doc_chunk_idx" UNIQUE ("document_id", "chunk_index"),
  CONSTRAINT "fk_chunks_document_user" FOREIGN KEY ("document_id", "user_id")
              REFERENCES "documents"("id", "user_id") ON DELETE CASCADE
);
CREATE INDEX IF NOT EXISTS "idx_document_chunks_doc_user" ON "document_chunks"("document_id", "user_i
d");
CREATE INDEX IF NOT EXISTS "idx_document_chunks_search" ON "document_chunks" USING gin("search_vecto
r");

-- 6. 業務資料表: summaries (複合外鍵連線 documents)
CREATE TABLE IF NOT EXISTS "summaries" (
  "id" uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
  "document_id" uuid NOT NULL,
  "user_id" uuid NOT NULL,
  "overall_summary" text NOT NULL,
  "chapter_points" jsonb NOT NULL DEFAULT '[]'::jsonb,
  "keywords" jsonb NOT NULL DEFAULT '[]'::jsonb,
  "key_concepts" jsonb NOT NULL DEFAULT '[]'::jsonb,
  "created_at" timestamp NOT NULL DEFAULT NOW(),
  "updated_at" timestamp NOT NULL DEFAULT NOW(),
  CONSTRAINT "uq_summaries_doc_user" UNIQUE ("document_id", "user_id"),
  CONSTRAINT "fk_summaries_document_user" FOREIGN KEY ("document_id", "user_id")
              REFERENCES "documents"("id", "user_id") ON DELETE CASCADE
);

-- 7. 業務資料表: notes (複合外鍵連線 courses 與 documents，含樂觀鎖版本)
CREATE TABLE IF NOT EXISTS "notes" (
  "id" uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
  "user_id" uuid NOT NULL REFERENCES "users"("id") ON DELETE CASCADE,
  "course_id" uuid,
  "document_id" uuid,
  "title" varchar(255) NOT NULL,
  "content" text NOT NULL,
  "version" integer NOT NULL DEFAULT 1,
```

```
  "created_at" timestamp NOT NULL DEFAULT NOW(),
  "updated_at" timestamp NOT NULL DEFAULT NOW(),
  CONSTRAINT "uq_notes_id_user" UNIQUE ("id", "user_id"),
  CONSTRAINT "fk_notes_course_user" FOREIGN KEY ("course_id", "user_id")
              REFERENCES "courses"("id", "user_id") ON DELETE SET NULL,
  CONSTRAINT "fk_notes_document_user" FOREIGN KEY ("document_id", "user_id")
              REFERENCES "documents"("id", "user_id") ON DELETE SET NULL
);
CREATE INDEX IF NOT EXISTS "idx_notes_user_course" ON "notes"("user_id", "course_id");

-- 8. 業務資料表: chat_sessions (建立複合 UNIQUE)
CREATE TABLE IF NOT EXISTS "chat_sessions" (
  "id" uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
  "user_id" uuid NOT NULL REFERENCES "users"("id") ON DELETE CASCADE,
  "course_id" uuid,
  "title" varchar(255) NOT NULL,
  "created_at" timestamp NOT NULL DEFAULT NOW(),
  "updated_at" timestamp NOT NULL DEFAULT NOW(),
  CONSTRAINT "uq_chat_sessions_id_user" UNIQUE ("id", "user_id"),
  CONSTRAINT "fk_chat_sessions_course_user" FOREIGN KEY ("course_id", "user_id")
              REFERENCES "courses"("id", "user_id") ON DELETE SET NULL
);
CREATE INDEX IF NOT EXISTS "idx_chat_sessions_user" ON "chat_sessions"("user_id");

-- 9. 業務資料表: chat_messages (複合外鍵連線 chat_sessions)
CREATE TABLE IF NOT EXISTS "chat_messages" (
  "id" uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
  "session_id" uuid NOT NULL,
  "user_id" uuid NOT NULL,
  "role" varchar(20) NOT NULL CHECK (role IN ('user', 'assistant')),
  "content" text NOT NULL,
  "sources" jsonb NOT NULL DEFAULT '[]'::jsonb,
  "created_at" timestamp NOT NULL DEFAULT NOW(),
  CONSTRAINT "fk_chat_messages_session_user" FOREIGN KEY ("session_id", "user_id")
              REFERENCES "chat_sessions"("id", "user_id") ON DELETE CASCADE
);
CREATE INDEX IF NOT EXISTS "idx_chat_messages_session" ON "chat_messages"("session_id");
```

- **注意規範**：Node.js 原生 `pg` 套件在執行 UPDATE 判斷時，一律使用 `result.rowCount`（小寫 row，大寫 Count）。

---

14. 中文全文搜尋與 Search Vector 更新重建機制（P1-6 規範）

14.1 檢索原理與分詞管道

1. **分詞演算法**：中文文字序列進行 2-gram 雙字切片（如「資料庫正規化」-> `資料 料庫 庫正 正規 規化`），英數字元轉為小寫單詞，以空格串接為 Tokens 字串。
2. **入庫產生**：新增 Chunk 時呼叫 `to_tsvector('simple', $tokens)` 寫入 `search_vector` 欄位。
3. **檢索三層次**：

- **OR 相關性搜尋（主要模式）**：以 `|` 串接 tokens，`ts_rank_cd` 降冪排序。
- **AND 包含搜尋**：以 `&` 串接，僅保證所有 Token 存在，**不保證相鄰性與順序**。

- **Phrase Search（精確詞組）**：PostgreSQL `<->` 相鄰運算子與 SQL `AND content LIKE '%' || $phrase || '%'` 二階段雙重比對。

14.2 Search Vector 更新與重建機制

- **更新情境**：
- 新增/替換 Chunk 時，隨 INSERT 語法即時計算 `search_vector`。
- 當分詞演算法升級（例如調整切詞規則或停用詞表）時，執行後端維護腳本 `npm run search:reindex`：

透過 Cursor 批次讀取 `document_chunks`，重新生成分詞字串並 UPDATE `search_vector`，重建期間不鎖定讀取。

---

15. 個人筆記系統健全性（樂觀鎖定與狀態碼統整）

1. **筆記編輯器**：React 原生輕量 Markdown 編輯，支援標題、粗斜體、清單、程式碼區塊與即時預覽。
2. **樂觀鎖更新判定（Node.js pg `result.rowCount`）**：

```
UPDATE notes
SET title = $1, content = $2, version = version + 1, updated_at = NOW()
WHERE id = $3 AND user_id = $4 AND version = $5
RETURNING version, updated_at;
```

- 若 `result.rowCount === 1`：回傳 `HTTP 200 OK`，附帶最新 `version` 與 `updated_at`。
- 若 `result.rowCount === 0`：後端立即執行查核 SQL：

```
SELECT id, user_id, version FROM notes WHERE id = $3;
```

- 查無記錄或 `user_id !== req.session.userId`：回傳 `HTTP 404 Not Found`（或 403 Forbidden）。
- 記錄存在但 `version !== $5`：回傳 `HTTP 409 Conflict`。

1. **草稿永不丟失與路由切換守衛**：收到 409 或網路錯誤時編輯器文字絕不被覆蓋清空，彈出版本比對對話框；離開頁面時若 `isDirty` 主動執行 `flushSave()` 等待 200 OK 方切換。

---

16. Render.com 單一服務部署與環境配置

1. **單一服務**：Build Command: `npm run build`，Start Command: `node server/index.js`，監聽 `process.env.PORT || 3000`，綁定 `0.0.0.0`。
2. **PostgreSQL SSL**：正式環境預設啟用憑證驗證（`rejectUnauthorized: true`），支援 `PG_CA_CERT` 環境變數。
3. **優雅關閉**：監聽 `SIGTERM` / `SIGINT`，等待進行中任務，關閉 DB 連線池後退出。
4. **路由隔離**：`/api/*` 未匹配路由一律回傳 JSON 404，SPA Fallback 嚴格僅回退前端 HTML。

---

17. 完整測試矩陣（44 項具體測試驗收標準）

保留原有 T01~T36，新增 T37~T44。**由於目前處於 Phase 1 需求規劃階段，測試狀態全數明確標註為：`Planned / Not Yet Executed`**：

編號	測試項目	測試前提	測試步驟	預期結果	通過標準	測試階段
T01	**原始檔案持久化保存**	使用者登入	上傳一份 PDF 講義	解析完成後，伺服器儲存路徑仍能讀取原始檔案	原始檔案未被刪除，檔案雜湊相符	Phas 3
T02	**原始檔案安全下載**	檔案已上傳	點擊下載按鈕發送請求	回傳檔案串流，Header 附帶正確 RFC 5987 檔名	能下載完整檔案，內容無損	Phas 3
T03	**原始檔案在線預覽**	PDF 講義已上傳	於 Split View 開啟預覽	瀏覽器安全預覽 PDF，CSP 沙箱生效	正確呈現預覽，無腳本執行風險	Phas 3
T04	**重啟後檔案仍存在**	檔案已上傳保存	重啟 Express 伺服器	重新請求 `/download` 與 `/preview`	檔案正常讀取，無 404 錯誤	Phas 3
T05	**備份與還原驗證**	DB 有檔案資料	執行備份與模擬災難還原	恢復 DB 與檔案目錄後資料完整可查	所有檔案與關聯筆記皆可存取	Phas 5
T06	**任務 Retry 驗證**	檔案狀態為 failed	呼叫 `/api/documents/:id/retry`	重新分派任務，利用既有原始檔重跑解析	狀態成功轉為 completed，Chunks 入庫	Phas 3
T07	**任務 Reupload 驗證**	原始檔損毀/遺失	呼叫 `/reupload` 上傳新檔	重新驗證檔案，覆蓋舊檔並重新解析	舊 Chunks 清除，新 Chunks 正確入庫	Phas 3
T08	**Reprocess 原子替換**	檔案 completed	呼叫 `/reprocess` 升級規則	在單一 DB 交易內刪除舊 Chunks 插入新 Chunks	若中途失敗立即回滾，舊資料不遺失	Phas 3
T09	**DOCX 深度驗證**	上傳偽造 DOCX	檢查 ZIP 內部結構	缺少 `word/document.xml` 立即報錯拒絕	偽造檔案被攔截，回傳 400	Phas 3
T10	**XLSX 多工作表解析**	含 3 個 Sheet 之 Excel	執行上傳解析	3 個工作表文字均擷取，標註 `sheetName`	搜尋可定位各工作表與列號	Phas 3
T11	**CSV 多編碼相容**	Big5 編碼之 CSV	執行上傳解析	自動嗅探編碼並以正確字集解碼入庫	中文字元無亂碼，列號定位精確	Phas 3
T12	**PPTX 投影片定位**	上傳含多頁簡報	執行上傳解析	保留 `slideNumber` 與 `slideTitle`	檢索結果精準標註投影片編號	Phas 3

編號	測試項目	測試前提	測試步驟	預期結果	通過標準	測試階段
T13	**PDF 頁碼定位**	上傳 20 頁 PDF	執行上傳解析	保留各頁文字與 `pageNumber`	檢索結果精確對應講義頁碼	Phas 3
T14	**大檔案串流下載**	15MB 講義	呼叫 `/download`	以 Stream Pipeline 傳輸，記憶體無暴增	下載流暢，伺服器記憶體平穩	Phas 3
T15	**ZIP Bomb 防禦**	上傳高壓縮比 ZIP	檢查未壓縮大小超過 100MB	監控觸發，立即中止連線並刪除暫存	伺服器無崩潰，回傳 400	Phas 3
T16	**XXE 外部實體防禦**	XML 含外部 DTD	執行 Office 解析	禁止載入外部實體，阻斷讀檔	無內部機密檔案洩漏	Phas 3
T17	**SSRF 禁止跳轉**	Base URL 回傳 302	代理請求該 Base URL	`maxRedirections: 0` 觸發，拒絕跳轉	回傳 SSRF 錯誤，無跳轉連線	Phas 3
T18	**SSRF IP Pinning**	輸入動態 DNS 域名	解析 A/AAAA 記錄比對黑名單	釘選安全 IP，連線至已驗證 IP	防禦 DNS Rebinding 成功	Phas 3
T19	**跨使用者檔案下載**	使用者 A 檔案 ID	使用者 B 發送下載請求	後端查核 `user_id` 不符，回傳 404	成功阻擋越權存取（IDOR）	Phas 3
T20	**跨使用者 RAG 隔離**	使用者 A 講義	使用者 B 進行 AI 問答	檢索 SQL 強制 `WHERE user_id = $userId`	使用者 B 絕無可能檢索到 A 的檔案	Phas 3
T21	**Worker 租約過期重整**	Worker 離線逾時	模擬實例崩潰超過 3 分鐘	自檢程式安全更新狀態為 failed 提示重試	租約正常失效，無死鎖任務	Phas 3
T22	**舊 Worker 寫入防護**	租約已由新實例接管	舊實例嘗試 COMMIT 寫入	`WHERE worker_id = $old` 匹配 0 筆自動回滾	舊實例未覆蓋新實例資料	Phas 3
T23	**筆記樂觀鎖成功**	正常編輯筆記	自動儲存送出相同版本號	影響列數為 1，回傳 200 與新版本號	版本號遞增，內容正常儲存	Phas 3
T24	**筆記樂觀鎖衝突**	傳入過期版本號	模擬兩端同時修改	影響列數為 0，後端查核回傳 409 Conflict	本地草稿保留，彈出版本衝突	Phas 3
T25	**筆記路由守衛阻擋**	筆記處於 isDirty	嘗試切換至另一篇筆記	觸發路由守衛，等待儲存成功後方切換	未儲存內容絕不丟失	Phas 3

編號	測試項目	測試前提	測試步驟	預期結果	通過標準	測試階段
T26	**Markdown XSS 防禦**	檔案含惡意 `<script>`	渲染預覽與 AI 回覆	`DOMPurify` 淨化所有危險標籤	無惡意腳本執行	Phas 3
T27	**API Key 日誌脫敏**	發送 AI 代理請求	檢查伺服器全域 Log 輸出	標頭與 Body 之 Key 均遮蔽為 `sk-***`	日誌中絕無真實金鑰留存	Phas 3
T28	**CSRF 標頭防禦**	跨站發起偽造 POST	無攜帶 `X-CSRF-Token`	後端中介層拒絕請求，回傳 403 Forbidden	CSRF 攻擊被成功攔截	Phas 3
T29	**帳號刪除級聯清理**	使用者執行刪除帳號	觸發 ON DELETE CASCADE	業務表全數清理，持久化實體檔案非同步刪除	無任何殘留孤兒檔案與資料	Phas 4
T30	**Render 部署健康檢查**	部署至 Render Web	呼叫 `/api/health` 端點	回傳 HTTP 200 與 DB 連線池正常狀態	雲端服務正常啟動，路由健全	Phas 5
T31	**雙 Worker 同時認領**	多實例啟動佇列	兩 Worker 同時認領同一任務	`FOR UPDATE SKIP LOCKED` 互斥生效，各得其一	任務絕不重複指派，無雙重解析	Phas 3
T32	**舊 Worker 覆蓋新結果**	Worker 延遲與重解析	舊 Worker 延遲嘗試寫入 Chunks	Fencing Token 比對不符，寫入被拒絕回滾	舊解析結果絕無覆蓋新結果	Phas 3
T33	**Reprocess 併發衝突**	同一檔案連點兩次 Reprocess	兩請求併發嘗試觸發解析	認領時推進 processing_version，前一請求失效	資料庫內容完全一致，無重複 Chunks	Phas 3
T34	**Storage 與 DB 不一致**	模擬磁碟檔案損毀/遺失	觸發解析前檢查 Checksum	檢驗不符立即拋出異常，標記 FILE_CORRUPTED	壞檔未進入解析，提示重新上傳	Phas 3
T35	**備份還原一致性演練**	模擬災難復原演練	還原 DB 與 Storage 檔案目錄	執行校驗腳本比對所有 Checksum	所有檔案與關聯筆記皆可正常存取	Phas 5
T36	**IPv4-mapped IPv6 SSRF**	傳入 `::ffff:127.0.0.1`	發起代理請求觸發驗證	後端解包還原為 IPv4 並命中黑名單	成功攔截變種 IPv6 繞過攻擊	Phas 3
T37	**跨使用者課程關聯防護**	使用者 A 建立講義	嘗試將 course_id 設為使用者 B 的課程	PostgreSQL 觸發複合外鍵違反異常	資料庫直接拒絕跨使用者關聯	Phas 2
T38	**跨使用者筆記文件關聯**	使用者 A 建立筆記	嘗試將 document_id 設為使用者 B 的檔案	PostgreSQL 觸發複合外鍵違反異常	資料庫直接拒絕跨租戶關聯	Phas 2

編號	測試項目	測試前提	測試步驟	預期結果	通過標準	測試階段
T39	**Worker 崩潰重試上限**	模擬解析連續拋出異常	Worker 依序執行 3 次失敗重試	第 3 次失敗後，retry_count=3，停止被認領	任務停留在 failed，無無限循環	Phas 3
T40	**退避期間重啟不提前認領**	失敗任務進入退避期	重啟 Express 伺服器	認領 SQL 比對 next_retry_at <= NOW()	退避到期前絕不提前認領	Phas 3
T41	**Reupload 解析失敗保護**	正常檔案已完成	上傳損壞新檔執行 Reupload	新檔解析拋出異常，交易回滾	舊檔案、舊 Chunks 完好無損	Phas 3
T42	**TXT / MD 定位 Schema**	上傳純文字與 Markdown	驗證生成的 source_location	通過 JSON Schema 驗證，具備正確行號	檢索結果精確標記行號與標題	Phas 3
T43	**Parser Stream 大檔處理**	上傳 10MB CSV / TXT	透過 getStream() 串流解析	記憶體增長在 50MB 內，無 OOM	伺服器記憶體平穩，解析成功	Phas 3
T44	**Chunk 唯一性約束防護**	模擬併發寫入同 index	嘗試 INSERT 相同 document_id 與 chunk_index	觸發 uq_chunks_doc_chunk_idx 唯一衝突異常	資料庫層級拒絕重複 Chunk	Phas 2

18. Phase1v5.2 → Phase1v5.3 修改紀錄表（P0 / P1 修正對照）

優先級	原本設計 (Phase1v5.2)	修改後設計 (Phase1v5.3)	修改原因與技術依據	修改章節	是否需實測
P0-1	`processing_version` 遞增時機在第 8 章與第 9 章不一致	**統一由 Worker 原子認領操作作為唯一遞增點**，提交時強制校驗，舊 Worker 寫入回滾	消除版本生命週期矛盾，確保 Fencing Token 世代絕對唯一，封死舊 Worker 覆蓋	第 8, 9 章	是
P0-2	外鍵僅檢查單一 ID，存在跨租戶錯誤關聯風險	**父表宣告 `(id, user_id)` 複合 UNIQUE，子表全面實作複合 FOREIGN KEY**	在 PostgreSQL 資料庫約束層級徹底封死跨使用者越權綁定（IDOR）	第 13 章	是
P0-3	Worker 狀態未含 cancelled，缺少退避時間持久化，恐致無限重試與提前認領	**新增 `cancelled` 狀態，引入 `next_retry_at` 持久化指數退避**，重試上限 3 次	避免反覆崩潰無限重試，確保伺服器重啟後依然嚴格遵守退避時間	第 9, 13 章	是
P1-1	Source Location Schema 缺少 TXT / MD 定義	**補齊 `txt` 與 `md` 專用 Schema**（`startLine`, `endLine`, `headingPath` 等）	完備第一版正式支援格式之結構化定位契約	第 6 章	是
P1-2	Parser 介面僅支援 Buffer，大檔易引發 OOM	**升級為 `inputHandle` 介面，支援 `getStream()` 串流讀取**	避免數十 MB 大檔整檔讀入記憶體，確保記憶體穩定性	第 5 章	是
P1-3	Reupload 流程缺乏原子替換與失敗保護	**定義 Reupload 五階段無損原子替換流程**，新檔成功前絕不刪除舊資料	防止重新上傳解析失敗時丟失既有可用資料	第 10 章	是
P1-4	儲存層與資料庫缺乏明確補償機制	**定義上傳補償、刪除補償與每週孤兒檔案掃描排程**	解決分散式檔案系統與關聯式資料庫非單一事務的一致性挑戰	第 4 章	是
P1-5	`document_chunks` 缺乏唯一約束	**新增 `CONSTRAINT uq_chunks_doc_chunk_idx UNIQUE (document_id, chunk_index)`**	防止同一文件版本產生重複的 Chunk Index	第 13 章	是
P1-6	Search Vector 缺乏明確更新與重建機制	**明列 2-gram 切詞產生管線、參與欄位與排程重建維護腳本**	確保檢索索引架構落地可行，不留概念空白	第 14 章	是
P1-7	檔案限制僅有上傳大小，缺乏展開與資源限制	**統一制定全格式限制表**（解壓大小上限 100MB、壓縮比 20:1、逾時 60s 等）	全面封閉 ZIP Bomb 與資源耗盡型 DoS 攻擊	第 7 章	是
P1-8	備份缺乏明確復原目標	**定義 RPO (24h) 與 RTO (2h)**，制定加密與聯合演練策略	建立客觀可驗收之災難復原基準	第 4 章	是

優先級	原本設計 (Phase1v5.2)	修改後設計 (Phase1v5.3)	修改原因與技術依據	修改章節	是否需實測
P1-9	SSRF 缺乏允許網段原則與完整特殊位址解包	**補齊 IPv4 完整特殊網段與 IPv4-mapped IPv6 強制解包比對**	杜絕各類偽裝與轉換型私有位址滲透	第 12 章	是
P1-10	測試矩陣僅有 36 項	**擴充為 44 項完整測試矩陣（新增 T37~T44）**，全數標註 `Planned`	完整覆蓋複合外鍵、重試上限、退避重啟、Reupload 保護與串流解析	第 17 章	是

19. 最終自我審查清單（強制跨章節一致性檢核）

- [x] **1. Processing Version 一致性**：已統一於第 8 章流程圖、第 8.1 節 SQL、第 9.2 節認領 SQL 與第 17 章測試案例（T32, T33），**唯一在認領時遞增**。
- [x] **2. Worker 狀態一致性**：狀態機、DDL CHECK 約束、認領 SQL 與取消流程統一支援 `pending`, `processing`, `completed`, `failed`, `cancelled` 5 種狀態。
- [x] **3. 退避持久化一致性**：`next_retry_at` 已列入 DDL 欄位、認領 SQL 判斷條件與失敗更新 SQL，重啟後嚴格生效。
- [x] **4. 複合外鍵跨租戶隔離**：`courses`, `documents`, `chat_sessions` 已宣告 `(id, user_id)` 複合唯一約束，所有子表全面實作複合外鍵。
- [x] **5. Chunk 唯一性**：已宣告 `uq_chunks_doc_chunk_idx` 唯一約束。
- [x] **6. Source Location Schema**：已補齊 `txt` 與 `md` 結構定義，全數納入標準 JSON Schema。
- [x] **7. Parser 介面串流化**：已升級為 `inputHandle` 支援 `getStream()`，解決大檔記憶體問題。
- [x] **8. 測試矩陣與狀態**：完整收錄 T01~T44，**44 項測試狀態全數誠實標記為 `Planned / Not Yet Executed`，無任何虛假通過**。
- [x] **9. 既有架構無退化**：純 JS、React+Vite、Express、PostgreSQL+pg、無 ORM、無外部佇列、單一 Render 服務完全保留。
- [x] **10. 開發狀態聲明**：**明確確認截至目前為止，尚未開始任何實質專案程式碼開發**，工作目錄維持純淨。

二十、Phase 2 准入結論（Go / No-Go Decision）

- **審查結論**：**【APPROVED（准予進入 Phase 2）】**。
- **判定理由**：所有 P0 缺陷（版本覆蓋、任務認領競爭、跨租戶外鍵穿透）與 P1 缺陷（TXT/MD Schema、大檔串流、退避持久化、SSRF 解包）均已取得完備且經 PostgreSQL 與 Node.js 語法檢驗之工程解決方案，全文 20 章前後規格 100% 嚴密一致，具備立即展開資料庫 Migration 與專案骨架建置之充足條件！